Participantes: Edson David Garcia Huanca

1. Guía de Ejecución y Despliegue

haciendo el git pull del backend y frontend
última actualización: 12 de junio de 2026
En caso de actualizarlo una vez más se hará una release con el nombre: “ENTREGA - 12 JUNIO”

1.1. Inicialización del Backend

npm start (luego del npm install)

Deberia dar el siguiente resultado

```
> P1@1.0.0 start
> NODE_ENV=production node index.js

◇ injected env (4) from .env // tip: ⌘ custom filepath { path: '/custom/path/.env' }
◇ injected env (0) from .env // tip: ⌘ suppress logs { quiet: true }
Usuario administrador creado con éxito! (User: admin / Pass: admin)
Usuario organizador creado con éxito! (User: org / Pass: org)
```

He creado un user admin y un user organizador.
debido a que no he implementado el createSpecialUser por tiempo y lo tengo para que se pueda comprobar las funcionalidades segun el rol

1.1. Inicialización del FrontEnd

Para empezar he cambiado lo que hace “npm run dev” en package.json
de `"dev": "vite"` a `"dev": "vite --host",`
Por lo que he entendido sino hago esto no puedo entrar a la pagina ya que está en un container y su lógica de redes (localhost) es diferente a la del pc

Ejecutar “npm run dev” (dentro de la carpeta /frontend)

```
> frontend@0.0.0 dev
> vite --host

 VITE v8.0.16  ready in 736 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://172.17.0.2:5173/
  ➜  press h + enter to show help
```

Resumen del frontend

NAVEGACIÓN

Básicamente he hecho que tengas como 2 o 3 filas,
- la 1r fila es lo que puede ver/consultar todo el mundo
- la 2a fila es para manejar la sesión, depende de si hay sesión iniciada o no saldrán unas cosas o otras
- la 3r fila solo aparece con la sesión iniciada, son las opciones especiales que tiene cada tipo de usuario, los admins tienen sus cosas, los users las suyas y así.

Como no tiene ROUTE luego simplemente mencionar que con sesión iniciada hay una opción para hacer "logout" que elimina el user de localStorage

ROUTES

Debido a que todo pasa por Routes, pues explicaré cada una.

"Global":

Route path="/"

Pâgina principal, se ve siempre:
- si está logueado no se ve nada (me acabo de dar cuenta)
- si no esta logueado te muestra informacion y 2 botones que son lo mismo que podrías ver gracias al Router, pero no esta de mas.

Route path="/videogames"

Básicamente muestra la lista de juegos, el nombre y su descripción
Si no hay muestra un mensaje
Hay filtros para filtrar según lo que indica el recuadro

Route path="/tournaments"

Básicamente muestra la lista de torneos con un boton en cada torneo para ver en mas detalle ese torneo
Si no hay muestra un mensaje
Hay filtros para filtrar según lo que indica el recuadro
Al darle al botón te lleva a /tournaments/:id
"

"Semi Global":

Route path="/tournaments/:id"

Aqui puede acceder cualquiera, pero hay mas opciones dependiendo del rol del usuario
- User : si esta abierta la inscripción puede inscribirse y desuscribirse
- Organizer: puede modificar el torneo

"Sesión NO iniciada":

Route path="/login"

Es un formulario simple para intentar iniciar sesión
Guarda el token en el localStorage

Route path="/register"

Es un formulario simple para crear una cuenta (de role : user)
No inicia la sesión, luego de crearla hace falta hacer login

“Session iniciada”:

Route path="/profile"

La informacion del usuario, por como tenia el backend, simplemente se puede modificar el correo,

podria tambien la contrasenya, seria casi lo mismo que poner el correo, no lo hice porque en un entorno real no se pondria una opcion para canviar la contrasenya sin seguridad extra, que no hice

Users

Route path="/user/tournaments"

Solo para users

Lista de los torneos a los que el usuario esta inscrito

Organizadores

Route path="/tournaments/create"

Solo para organizadores

Un formulario para crear un torneo

como unica cosa que no me convence es que no da una lista de videojuegos disponibles

Route path="/organizer/tournaments"

Solo para organizadores

Lista de los torneos creados por el y en los que es un organizerSupport

Igualmente por ahora los organizerSupport no estan implementados pero ya estaria hecho

Admins

Route path="/createSpecialUser"

Solo para admins

Por ahora no hace nada, ja que no termine de implementarlo

Route path="/videogames/create"

Solo para admins

Un formulario simple para crear un videojuego

Estructura del proyecto

La mayoría, si no es que todas, las cosas que requieren de información por parte del usuario se proporcionan mediante un Form

Todas las llamadas al backend pasan por su Service correspondiente

Abajo hay información extra de decisiones que he tomado mientras hacía el frontend
Ordenado de la siguiente manera:
- Decisiones que afectan en general
- Decisiones que afectan más a una requisito en especifico

Añadido al backend:
- npm insall cors
En app.js

```
const cors = require("cors")
app.use(cors())
```

- Nueva función en utils para crear un usuario Admin y org se llama en App.js justo luego de iniciar la BDD (Hecho porque sino no podía probar las cosas que requieren Special Users)

- He decidido manejar el token en otro lado, he puesto que el export en el getToken para que se pueda simplemente exportar el getToken, ja que en la mayoría (cuando escribo esto) de servicios simplemente necesito el getToken y no el setToken, también los he puesto en un archivo aparte para que sea más fácil de manejar/controlar, en lugar de ponerlo en algún service específico

- He usado la IA en algunos sitios para que se vea mejor con style algunas cosas, como es meramente visual he pensado que es mejor ponerlo aunque no haya decidido del todo yo el style

- Me he dado cuenta que uso mucho la pestaña network para ver los errores que me envia el backend, y no he prestado atención a que el usuario reciba estos mensajes, tendria que adaptar los errores que obtengo para que los pueda ver/leer el usuario, aunque sea en lenguaje técnico.

- Me he dado cuenta que lo que tenía en el backend no era la manera la mejor manera de mostrar/hacer algunas cosas, como la info de torneo, debido a esto he añadido bastantes consultas SQL, modificado/añadido endpoint, con el simple objetivo de mejorar la información que recibe el frontend
Un ejemplo simple: torneo tiene el juego como Int/Id, con la nueva consulta SQL sale el nombre del juego directamente.
Estos cambios en el backend no tienen tests específicos para ellos.

Perfil d’usuari

- Debido a que tengo que hacer esto: *L'aplicació ha de permetre visualitzar el perfil de qualsevol usuari a partir del seu nom d’usuari*;

En el backend:

en /model/users.js

getUserByUsername; daba todo, le he quitado la contraseña (login usaba esto)

getAllUserByUsername: Añadido, da todo con la contraseña (login usa esto ahora)

en controllers/users.js

```
usersRouter.get("/:id",  →   usersRouter.get("/id/:id",
```

Añadido:

```
usersRouter.get("/username/:username"
```

Quería unificarlos pero si el nombre del usuario es solo números daria conflictos, para no tocar mucho mas simplemente añadí el otro

- *Quan l’usuari auténtica visita el seu propi perfil, ha de tener disponible un formulario d'edició que li permeti modificar les seves dades.*

Debido a esto he creado

```
usersRouter.patch("/username/:username"
```

De momento solo se puede modificar el correo, debido a que en el backend solo hay email y password para modificar y password no me parece bien poder modificarlo ahí

Visualització de videojocs

- *La llista ha de presentar, com a mınim, el nom i la descripció de cada videojoc. S’ha d’oferir un filtre textual que permeti cercar videojocs pel nom, i opcionalment per parts de la descripci´o.*

Simplemente para aclarar, soy consciente de que el buscador actualiza la lista de juegos en base a lo que obtuvo al principio, no hace nuevas llamadas al backend con el filtro

Visualitzaci´o d’un torneig

Si l’usuari est`a autenticat com a jugador i les inscripcions del torneig estan obertes, la p`agina ha de mostrar un bot´o per inscriure-s’hi. Un cop inscrit, s’ha d’indicar visualment que ja hi participa

Añadido al backend

en /controller/tournaments.js

```
router.get("/:id/participants/:username"
```

Y una pequeña modificacion en una llamada a la BDD para que se pueda comprobar con el username sin necesidad del userId